

Here is a technical breakdown of **CVE-2025-26943** in the WordPress Easy Quotes plugin.

1. Why This Vulnerability Happens

This is an **unauthenticated blind SQL Injection** vulnerability affecting Easy Quotes versions **≤ 1.2.2** .

Root Cause:

The plugin fails to properly **sanitize and escape** user-supplied input before incorporating it directly into SQL queries. In WordPress, this typically occurs when developers:

- Use raw `$_GET` or `$_POST` parameters directly in SQL strings
- Fail to use WordPress's built-in `$wpdb->prepare()` method
- Forget to validate or cast input types (e.g., expecting an integer but receiving a string)

Because the input is not neutralized, an attacker can inject malicious SQL syntax—such as `UNION SELECT`, boolean-based conditions, or time-delay functions like `SLEEP()`—to manipulate the database query execution. Since it is **blind** SQL injection, the attacker infers data from subtle differences in application responses or timing delays rather than direct output .

Impact:

- Unauthorized database access
- Extraction of sensitive data (user credentials, site configuration)
- Potential site takeover
- Mass exploitation risk (CVSS 9.3)

2. How Developers Should Fix It

The fix requires three layers of defense:

Layer	Action
Input Validation	Strictly validate all user inputs. Use intval() for IDs, sanitize_text_field() for strings, and whitelist expected values.
Parameterized Queries	Always use \$wpdb->prepare() which properly escapes variables and prevents injection.
Least Privilege	Ensure the database user has only the minimum required permissions.

Specific fixes:

- Replace all raw SQL concatenation with prepared statements
- Apply absint() for numeric IDs expected from \$_GET/\$_POST
- Add nonces (CSRF tokens) for state-changing operations
- Update to version 1.2.3 or later, which contains the official patch

3. Secure Coding Example

Here is a vulnerable vs. secure implementation pattern commonly found in WordPress plugins:

✖ Vulnerable Code (DO NOT USE)

```
```php
```

```
// Vulnerable: Directly interpolating user input
```

```
function get_quote_by_id() {
 global $wpdb;

 $quote_id = $_GET['quote_id']; // Untrusted input

 $sql = "SELECT * FROM {$wpdb->prefix}easy_quotes WHERE id = $quote_id";

 $result = $wpdb->get_row($sql);

 return $result;
}
```

```
```
```

 Secure Code (BEST PRACTICE)

```
```php
```

```
function get_quote_by_id() {
 global $wpdb;

 // 1. Validate and sanitize input

 $quote_id = isset($_GET['quote_id']) ? absint($_GET['quote_id']) : 0;

 if ($quote_id <= 0) {
 return new WP_Error('invalid_id', 'Invalid quote ID provided.'); }

 // 2. Use prepared statements with $wpdb->prepare()

 $sql = $wpdb->prepare(

```

```
"SELECT * FROM {$wpdb->prefix}easy_quotes WHERE id = %d",
 $quote_id
);
```

```
$result = $wpdb->get_row($sql);
```

```
// 3. Verify result exists before using
```

```
if (null === $result) {
 return new WP_Error('not_found', 'Quote not found.');
```

```
}
```

```
return $result;
```

```
}
```

```
...
```

####  Secure Code for String Parameters

```
```php
```

```
function search_quotes_by_author() {
```

```
    global $wpdb;
```

```
    // Sanitize string input
```

```
    $author = isset($_GET['author']) ? sanitize_text_field($_GET['author']) : '';
```

```
    if (empty($author)) {
```

```
        return array();
```

```
    }
```

```
// Use %s placeholder for strings
```

```
$sql = $wpdb->prepare(  
    "SELECT * FROM {$wpdb->prefix}easy_quotes  
    WHERE author = %s  
    AND status = %s",  
    $author,  
    'published'  
);
```

```
return $wpdb->get_results($sql);
```

```
}
```

```
...
```

****Key Security Principles:****

- ****`\$wpdb->prepare()`****: Escapes all variables and creates parameterized queries
- ****`absint()`****: Forces integer values, eliminating injection via numeric fields
- ****`sanitize\_text\_field()`****: Strips dangerous characters from strings
- ****Input validation****: Reject unexpected data types or values before they reach the database

```
---
```

Summary

CVE-2025-26943 exists because the Easy Quotes plugin trusted user input without sanitization or parameterization. The fix is straightforward: ****never trust user input****, always ****validate and escape**** data, and use ****WordPress's `wpdb->prepare()`**** for all database interactions. If you are running this plugin, update to version ****1.2.3+**** immediately .

